

Geometry-based Methods in Vision

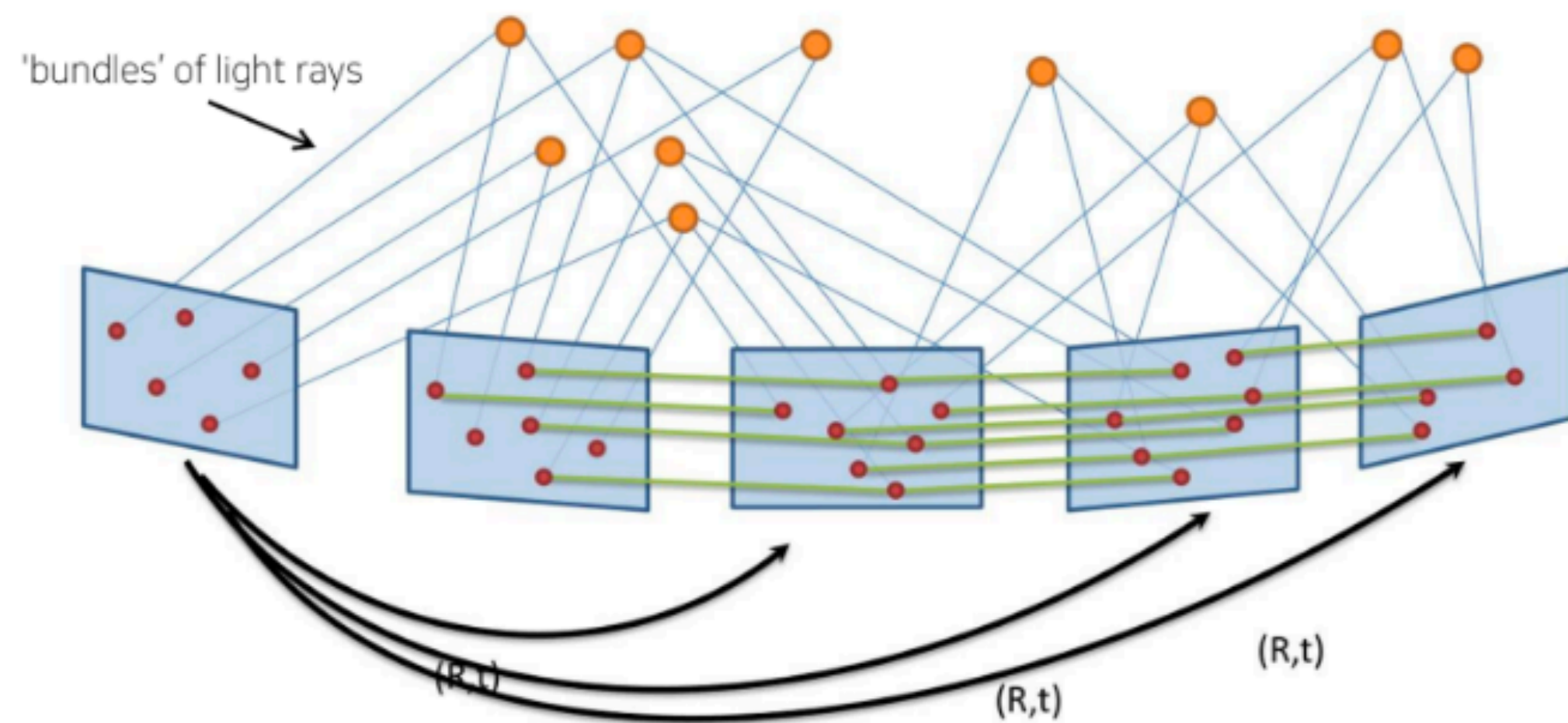
CMU 16-822, Fall 2024

Instructor: Shubham Tulsiani

Administrivia

- Project Proposals
 - *Please form teams (on canvas)*
 - *Due Saturday (**no late days**)*
- No office hours in Fall Break

Lecture 14a: Non-linear Least Squares



A reminder to ask questions!



Life is nice in Linear-land!

A

$m \times n$

x

$n \times 1$

$=$

0

$m \times 1$

$\min_x \|Ax\|^2 \text{ subject to } \|x\| = 1$

A

$m \times n$

x

$n \times 1$

$=$

b

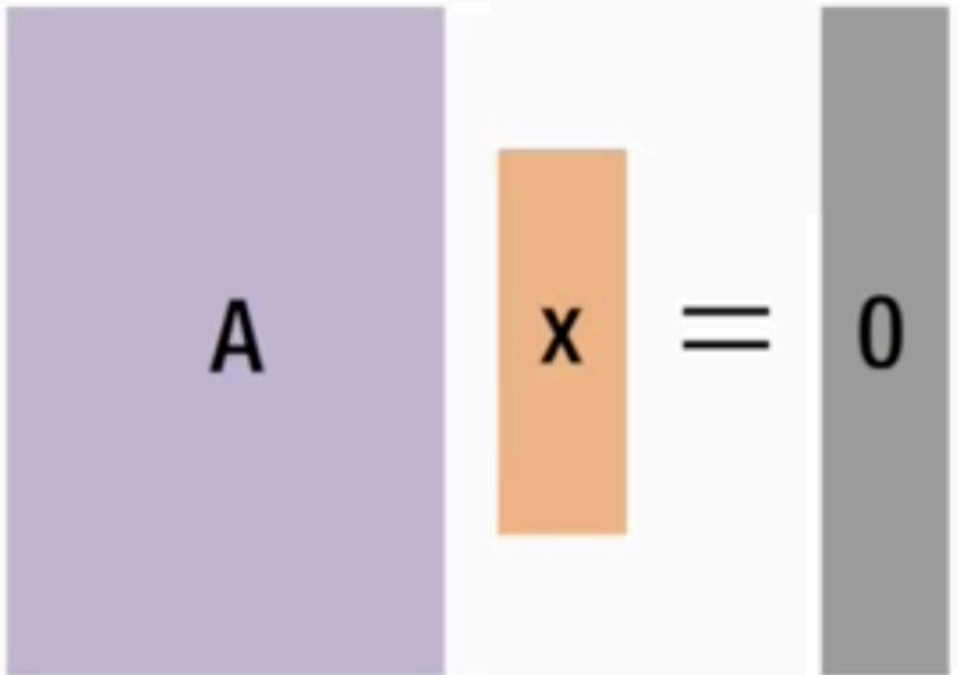
$m \times 1$

$\min_x \|Ax - b\|^2$

p_1, p_2, p_3, p_4 p'_1, p'_2, p'_3, p'_4 $\rightarrow H$ $\rightarrow P$ $\rightarrow \{(x_i, x'_i)\}$ F

Image credits: Jianbo Shi

Life is nice in Linear-land!

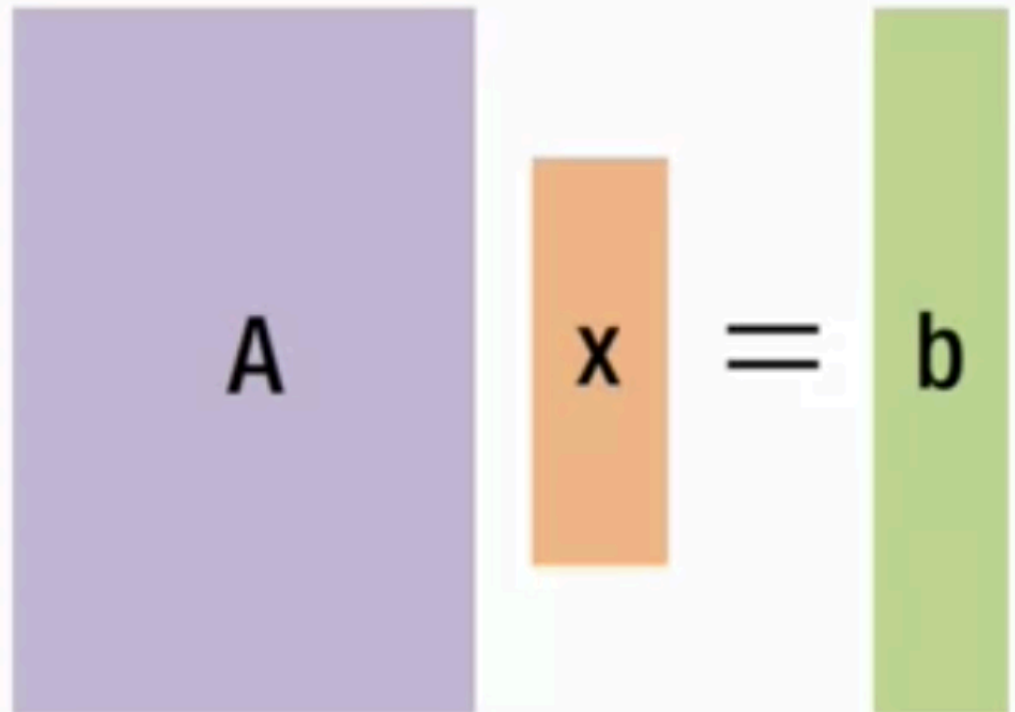


A x $=$ 0

$m \times n$ $n \times 1$ $m \times 1$

$\min_x \|Ax\|^2$ subject to $\|x\| = 1$

Solution: Singular Value(s)



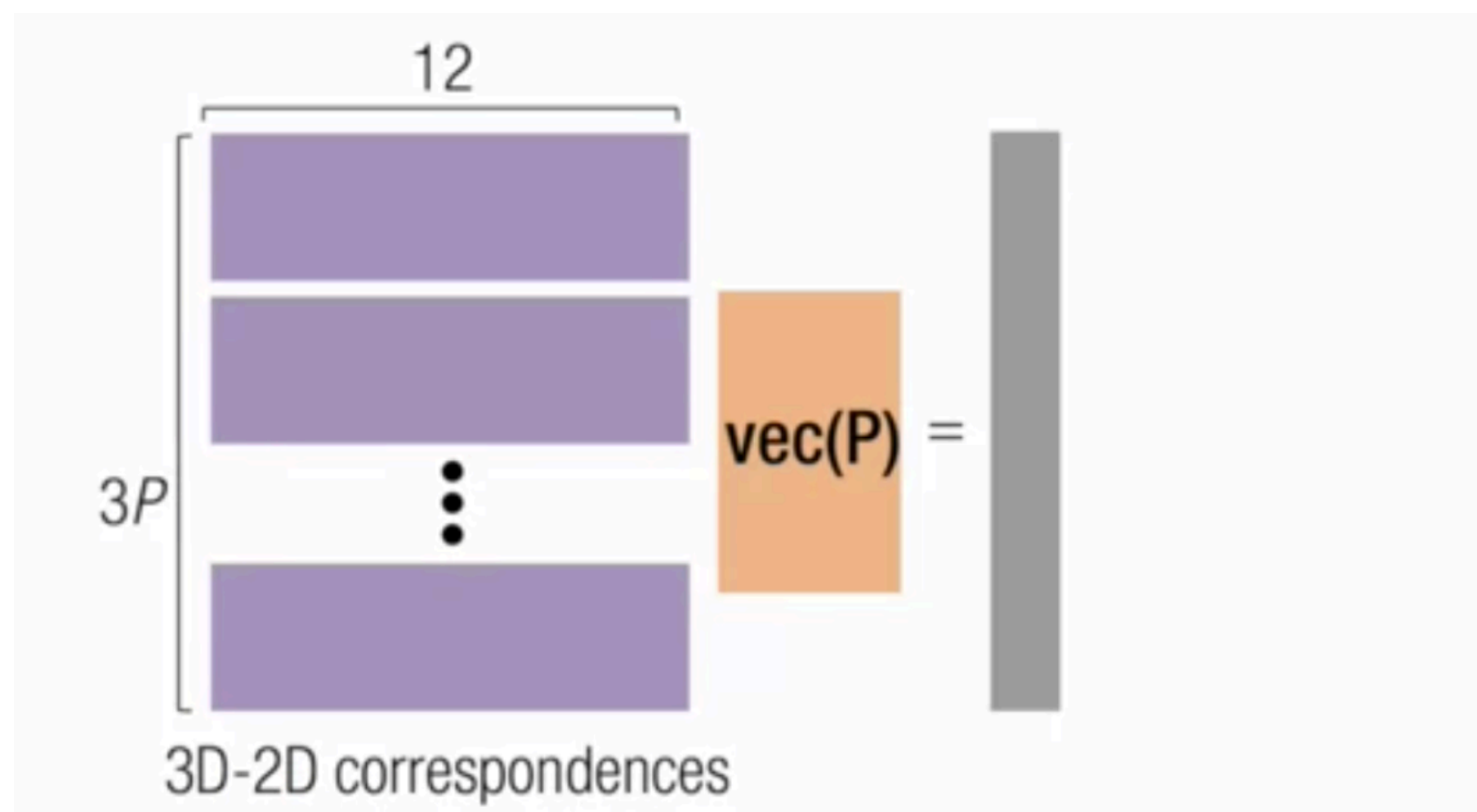
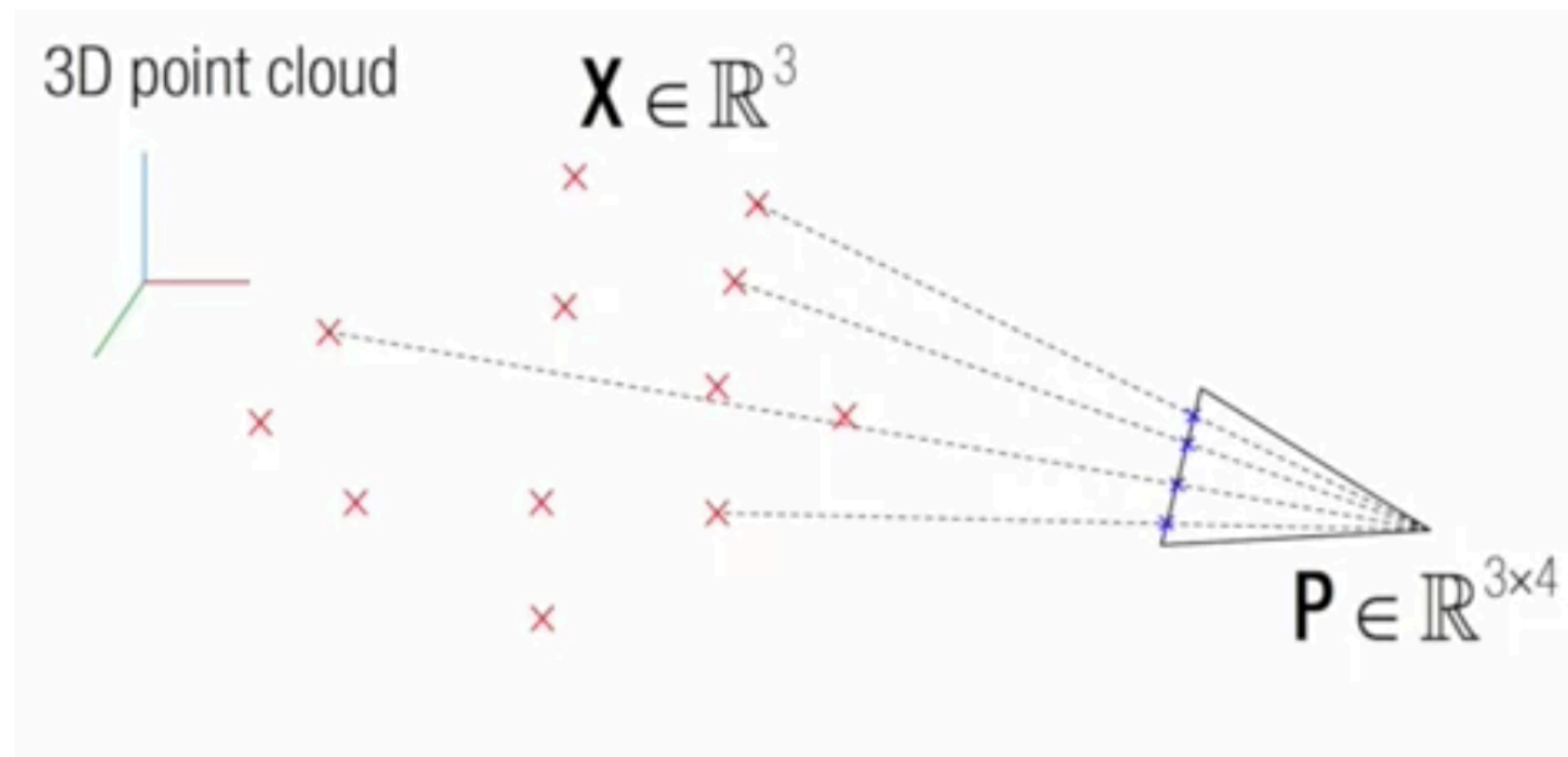
A x $=$ b

$m \times n$ $n \times 1$ $m \times 1$

$\min_x \|Ax - b\|^2$

$$x = (A^T A)^{-1} A^T b$$

But need to step into the realm of non-linearity



PnP problem — linear given general P (assuming algebraic error)

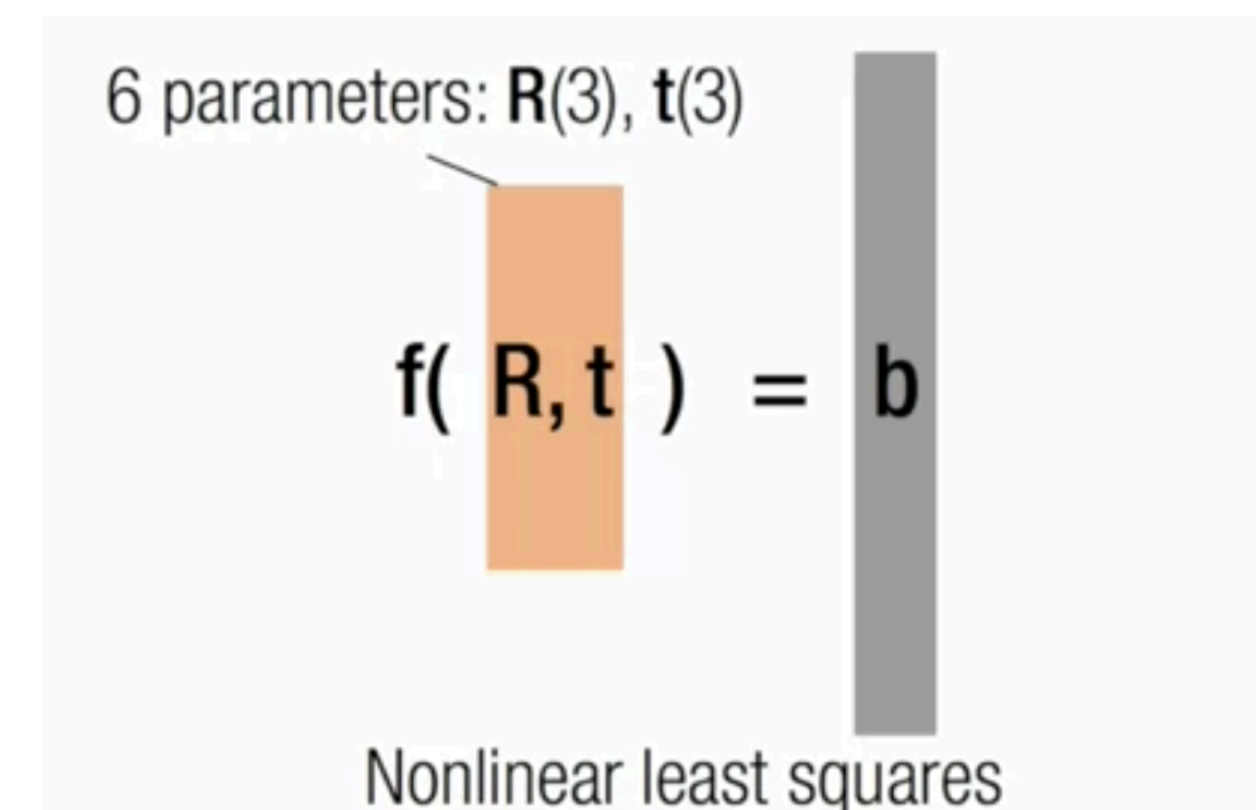
$$\mathbf{P}\mathbf{X}_i \times \mathbf{x}_i = \mathbf{0}$$

$$\begin{bmatrix} \mathbf{0} & -w\mathbf{X}^\top & y\mathbf{X}^\top \\ w\mathbf{X}^\top & \mathbf{0} & -x\mathbf{X}^\top \\ -y\mathbf{X}^\top & x\mathbf{X}^\top & \mathbf{0} \end{bmatrix} \mathbf{p} = \mathbf{0}$$

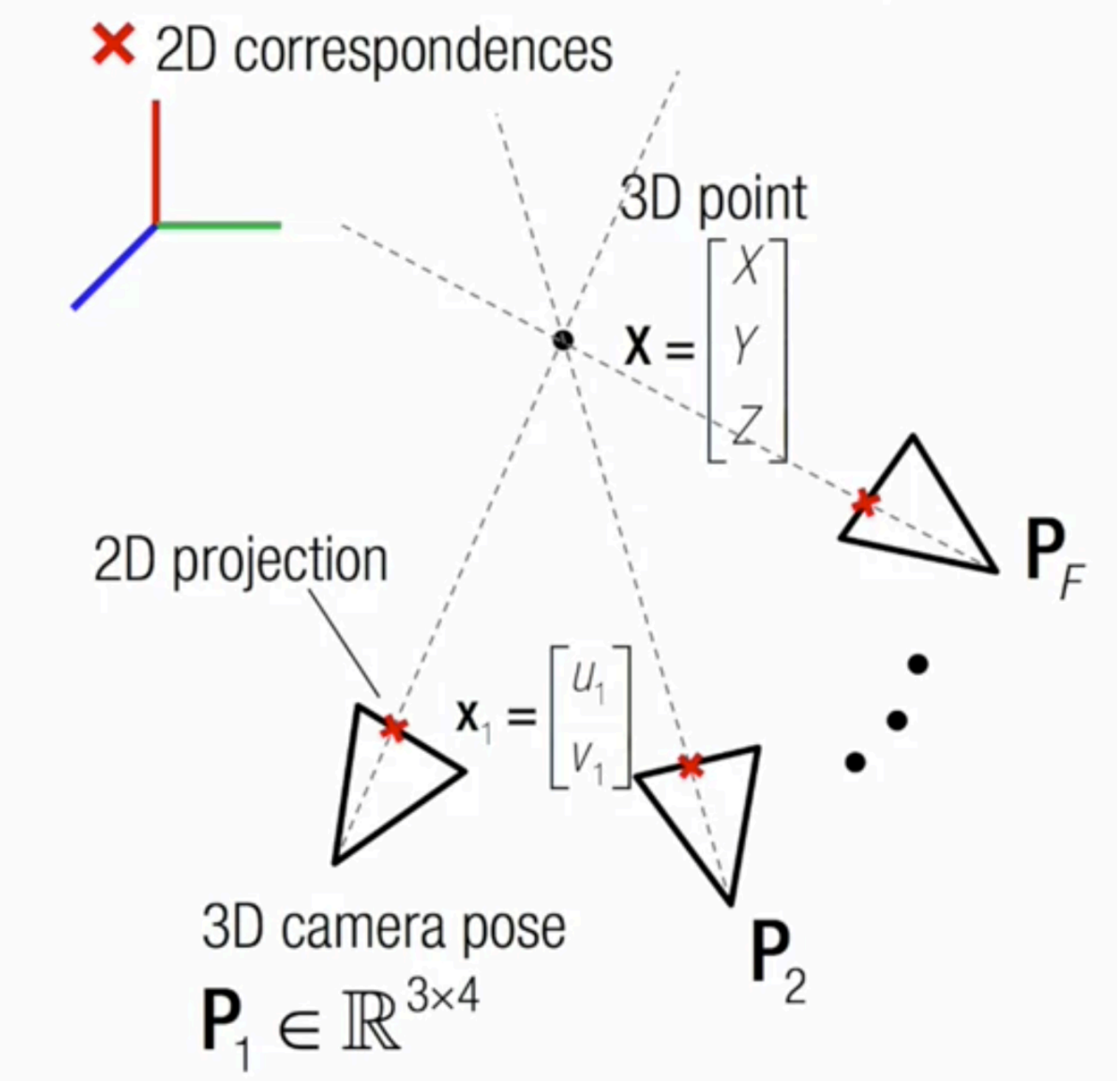
But non-linear in R, t i.e estimating extrinsics assuming known intrinsics (even if optimizing algebraic error!)

$$\mathbf{P} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \in \mathbb{R}^{3 \times 4}$$

where $\mathbf{R} \in \mathbf{SO}(3)$



But need to step into the realm of non-linearity



Triangulation from 2D correspondences

$$3F \begin{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ 1 \end{bmatrix}_x \mathbf{P}_1 \\ \begin{bmatrix} \mathbf{x}_2 \\ 1 \end{bmatrix}_x \mathbf{P}_2 \\ \vdots \\ \begin{bmatrix} \mathbf{x}_F \\ 1 \end{bmatrix}_x \mathbf{P}_F \end{bmatrix} \begin{bmatrix} \mathbf{X} \\ 1 \end{bmatrix} = \mathbf{0}$$

Linear in $\mathbf{X}$ assuming algebraic error

$$E_{\text{repro}} = (u - u_{\text{repro}})^2 + (v - v_{\text{repro}})^2$$

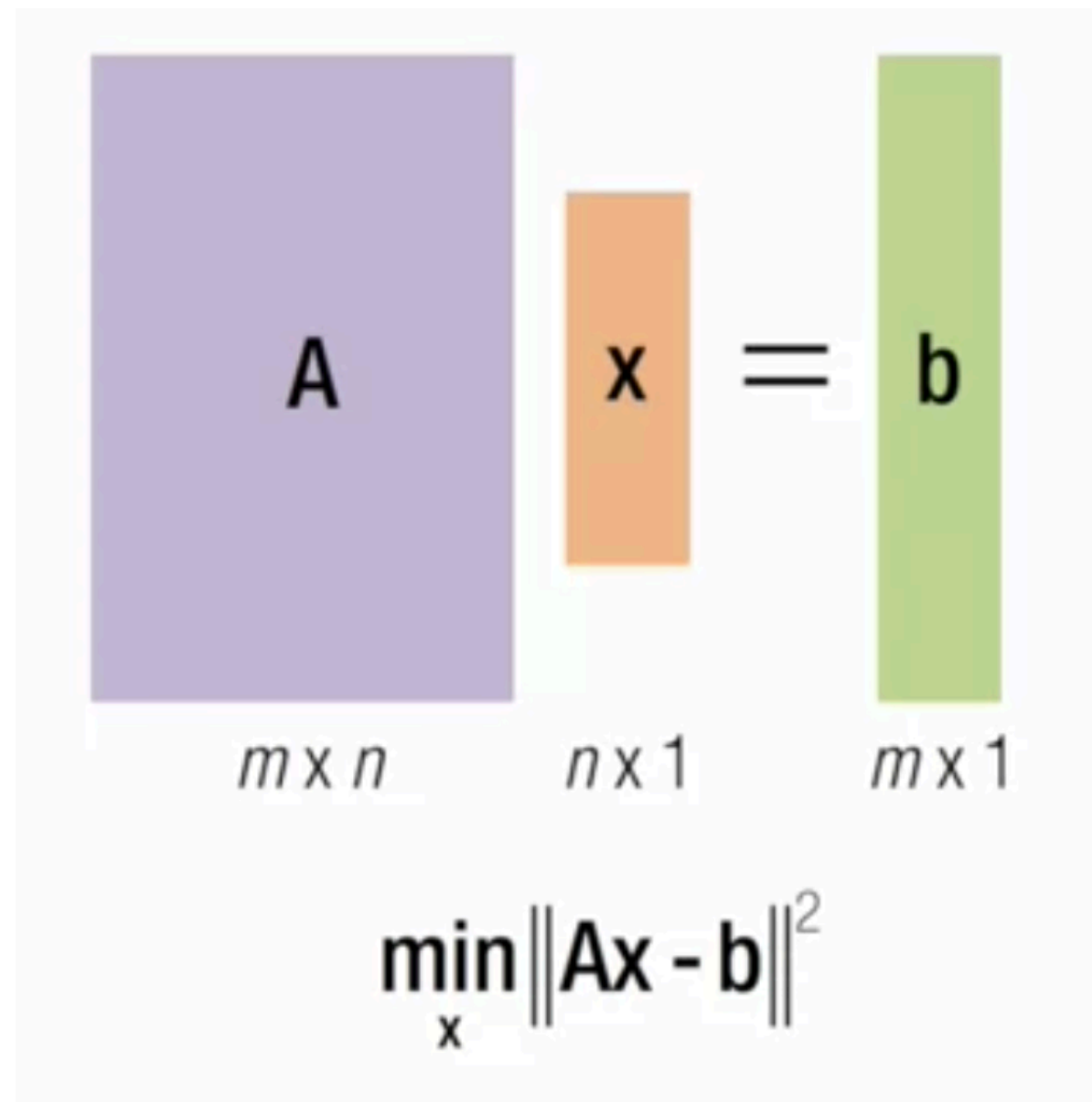
$$= \left(u - \frac{\mathbf{P}_1^1 \tilde{\mathbf{X}}}{\mathbf{P}_1^3 \tilde{\mathbf{X}}} \right)^2 + \left(v - \frac{\mathbf{P}_1^2 \tilde{\mathbf{X}}}{\mathbf{P}_1^3 \tilde{\mathbf{X}}} \right)^2$$

$$\min_{\mathbf{x}} \sum_{i=1}^F \left(u_i - \frac{\mathbf{P}_i^1 \tilde{\mathbf{X}}}{\mathbf{P}_i^3 \tilde{\mathbf{X}}} \right)^2 + \left(v_i - \frac{\mathbf{P}_i^2 \tilde{\mathbf{X}}}{\mathbf{P}_i^3 \tilde{\mathbf{X}}} \right)^2$$

Reprojection error minimization

Non-linear optimization given geometric reprojection error

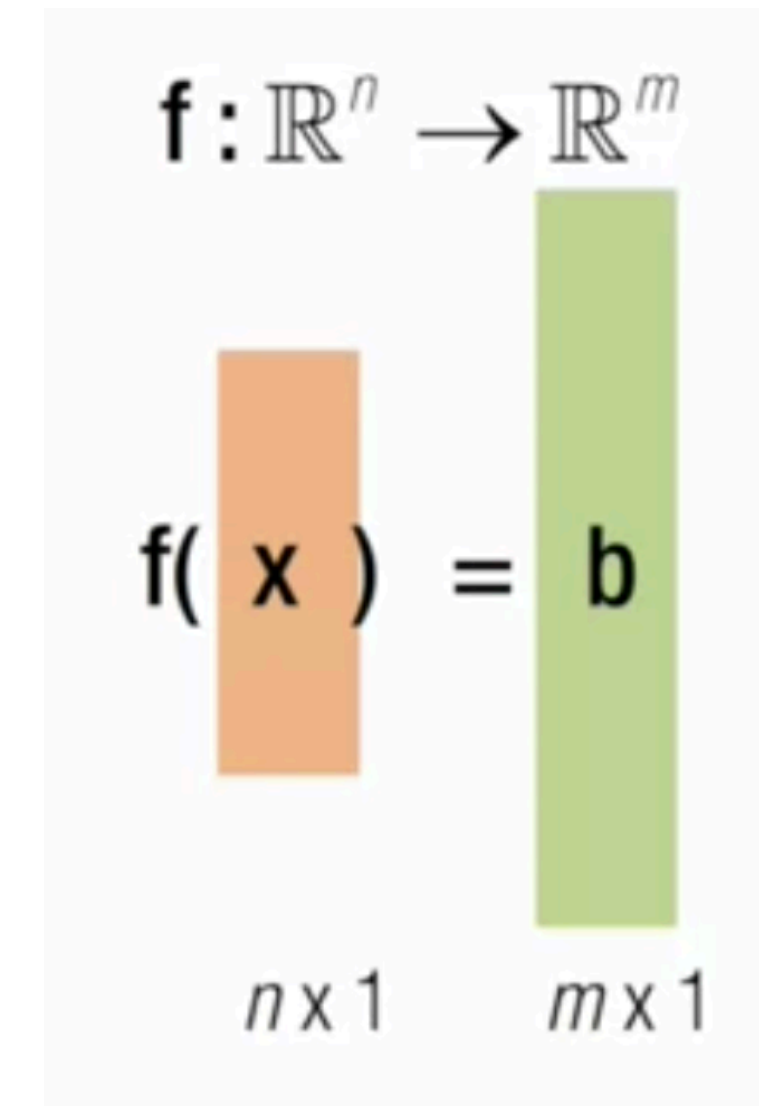
Least Squares vs Non-linear Least Squares



A diagram illustrating the linear least squares problem. It shows a purple rectangle labeled A with dimensions $m \times n$ below it. To its right is an orange rectangle labeled x with dimensions $n \times 1$ below it. Further right is a green rectangle labeled b with dimensions $m \times 1$ below it. An equals sign is placed between x and b . Below the rectangles, the minimization problem is stated: $\min_x \|Ax - b\|^2$.

$$\min_x \|Ax - b\|^2$$

$$\mathbf{x} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$



A diagram illustrating the non-linear least squares problem. At the top, it shows a function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$. Below this, an orange rectangle labeled $f(x)$ with dimensions $n \times 1$ below it is shown next to a green rectangle labeled b with dimensions $m \times 1$ below it. An equals sign is placed between $f(x)$ and b .

$$\min_{\mathbf{x}} \|f(\mathbf{x}) - \mathbf{b}\|^2$$

$$\min_{\mathbf{x}} \sum_m (f_m(\mathbf{x}) - b_m)^2$$

Equivalent form as sum-of-squares objective

Will develop techniques that work for arbitrary $\mathbf{f}$

Important that the objective is of the form of sum-of-squares!

A general approach to optimization

$$\min_{\mathbf{x}} C(\mathbf{x}) \quad (\text{C is a scalar cost function})$$

$$\text{Find } \mathbf{x}^* \text{ s.t. } \nabla_{\mathbf{x}} C = 0$$

(Linear) Least Squares:

$$C(\mathbf{x}) = (\mathbf{Ax} - \mathbf{b})^T (\mathbf{Ax} - \mathbf{b})$$

$$\nabla_{\mathbf{x}} C = 2\mathbf{A}^T \mathbf{Ax} - 2\mathbf{A}^T \mathbf{b}$$

$$\mathbf{x}^* = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$

A general approach to optimization

$$\min_{\mathbf{x}} C(\mathbf{x}) \quad (\text{C is a scalar cost function})$$

General cost function w. Taylor Expansion:

$$C'(\mathbf{y}) = C(\mathbf{x}_0) + \nabla_{\mathbf{x}_0}^T \mathbf{y} + \frac{1}{2} \mathbf{y}^T \mathbf{H}_{\mathbf{x}_0} \mathbf{y}$$

Find $\mathbf{y}^*$ s.t. $\nabla_{\mathbf{y}} C' = 0 \quad \mathbf{H}_{\mathbf{x}_0} \mathbf{y}^* = -\nabla_{\mathbf{x}_0}$

Newton's Method

$$\mathbf{y}^* \equiv \Delta \mathbf{x} = -(\nabla^2 C(\mathbf{x}))^{-1} \nabla C(\mathbf{x})$$
$$\mathbf{x} \leftarrow \mathbf{x} + \Delta \mathbf{x}$$

Repeat until
convergence

For a general cost function $\mathbf{C}$, need 2nd order Taylor expansion!

Can we follow this approach w/o computing Hessians?

Yes, for some objectives — non-linear least squares!

Non-linear Least Squares

$$\min_{\mathbf{x}} C(\mathbf{x}) \quad C(\mathbf{x}) = (f(\mathbf{x}) - \mathbf{b})^\top (f(\mathbf{x}) - \mathbf{b})$$

Taylor Expansion:
(of $\mathbf{f}$, not $\mathbf{C}$!)

$$\begin{aligned} f(\mathbf{x}_0 + \mathbf{y}) &= f(\mathbf{x}_0) + \frac{\partial f}{\partial \mathbf{x}}(\mathbf{x}=\mathbf{x}_0) \mathbf{y} \\ &\equiv f(\mathbf{x}_0) + \mathbf{J} \mathbf{y} \end{aligned}$$

$$\frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial \mathbf{f}_1}{\partial x_1} & \dots & \frac{\partial \mathbf{f}_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial \mathbf{f}_m}{\partial x_1} & \dots & \frac{\partial \mathbf{f}_m}{\partial x_n} \end{bmatrix}$$

$$C'(\mathbf{y}) = (\mathbf{J}\mathbf{y} + f(\mathbf{x}_0) - \mathbf{b})^\top (\mathbf{J}\mathbf{y} + f(\mathbf{x}_0) - \mathbf{b}) = (\mathbf{J}\mathbf{y} - \epsilon)^\top (\mathbf{J}\mathbf{y} - \epsilon)$$

$$\epsilon = \mathbf{b} - f(\mathbf{x}_0)$$

Find $\mathbf{y}^*$ s.t. $\nabla_{\mathbf{y}} C' = 0$

$$\mathbf{y}^* \equiv \Delta \mathbf{x} = (\mathbf{J}^\top \mathbf{J})^{-1} \mathbf{J}^\top \mathbf{e}$$

Non-linear Least Squares

$$\min_{\mathbf{x}} C(\mathbf{x}) \quad C(\mathbf{x}) = (f(\mathbf{x}) - \mathbf{b})^\top (f(\mathbf{x}) - \mathbf{b})$$

$$\boldsymbol{\epsilon} = \mathbf{b} - f(\mathbf{x})$$

$$\Delta \mathbf{x} = (\mathbf{J}^\top \mathbf{J})^{-1} \mathbf{J}^\top \mathbf{e}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \Delta \mathbf{x}$$

Gauss-Newton Method

Repeat until
convergence

$$\Delta \mathbf{x} = -(\nabla^2 C(\mathbf{x}))^{-1} \nabla C(\mathbf{x})$$

$$\nabla C(\mathbf{x}) = -\mathbf{J}^\top \mathbf{e}$$

Newton Method

Thus, Gauss-Newton similar to Newton's method, where we approximate the Hessian — possible due to sum-of-squares structure

Non-linear Least Squares

$$\min_{\mathbf{x}} C(\mathbf{x}) \quad C(\mathbf{x}) = (f(\mathbf{x}) - \mathbf{b})^T (f(\mathbf{x}) - \mathbf{b})$$

$$\epsilon = \mathbf{b} - f(\mathbf{x}) \quad \Delta \mathbf{x} = (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \mathbf{e} \quad \text{Gauss-Newton Method}$$

$$\Delta \mathbf{x} = (\mathbf{J}^T \mathbf{J} + \lambda I)^{-1} \mathbf{J}^T \mathbf{e} \quad \text{Marquardt Algorithm}$$

similar to gradient descent for high lambda

$$\Delta \mathbf{x} = (\mathbf{J}^T \mathbf{J} + \lambda \text{diag}(\mathbf{J}^T \mathbf{J}))^{-1} \mathbf{J}^T \mathbf{e} \quad \text{Levenberg-Marquardt Algorithm}$$

All above are iterative methods with differing update rules.

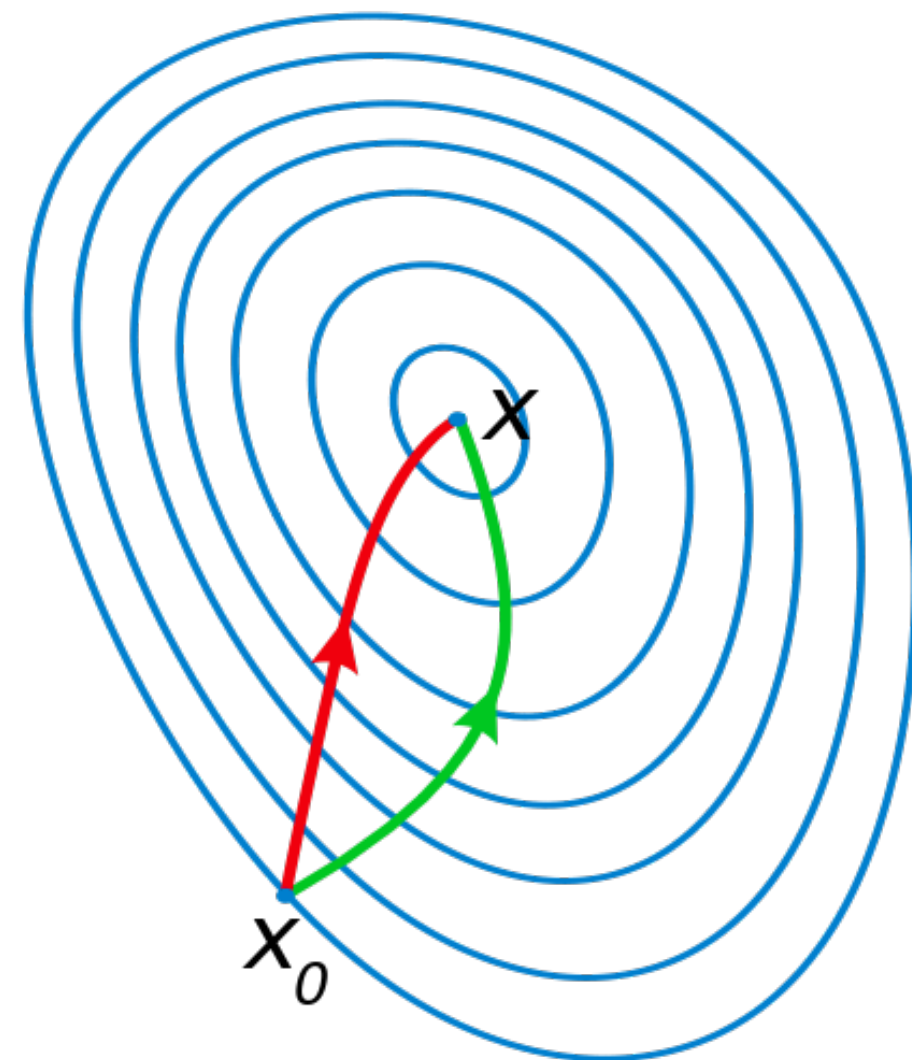
Step size can be an additional hyper-parameter

Non-linear Least Squares

$$\min_{\mathbf{x}} C(\mathbf{x}) \quad C(\mathbf{x}) = (f(\mathbf{x}) - \mathbf{b})^T (f(\mathbf{x}) - \mathbf{b})$$

$$\epsilon = \mathbf{b} - f(\mathbf{x}) \quad \Delta \mathbf{x} = (\mathbf{J}^T \mathbf{J} + \lambda \text{diag}(\mathbf{J}^T \mathbf{J}))^{-1} \mathbf{J}^T \mathbf{e}$$

Q: Why not just use gradient descent?



Efficient convergence!
(significant speed-ups and better convergence)

Non-linear Least Squares

$$\min_{\mathbf{x}} C(\mathbf{x}) \quad C(\mathbf{x}) = (f(\mathbf{x}) - \mathbf{b})^T (f(\mathbf{x}) - \mathbf{b})$$

$$\epsilon = \mathbf{b} - f(\mathbf{x}) \quad \Delta \mathbf{x} = (\mathbf{J}^T \mathbf{J} + \lambda \text{diag}(\mathbf{J}^T \mathbf{J}))^{-1} \mathbf{J}^T \mathbf{e}$$

Q: If LM etc. are so great, why do we even use gradient descent?

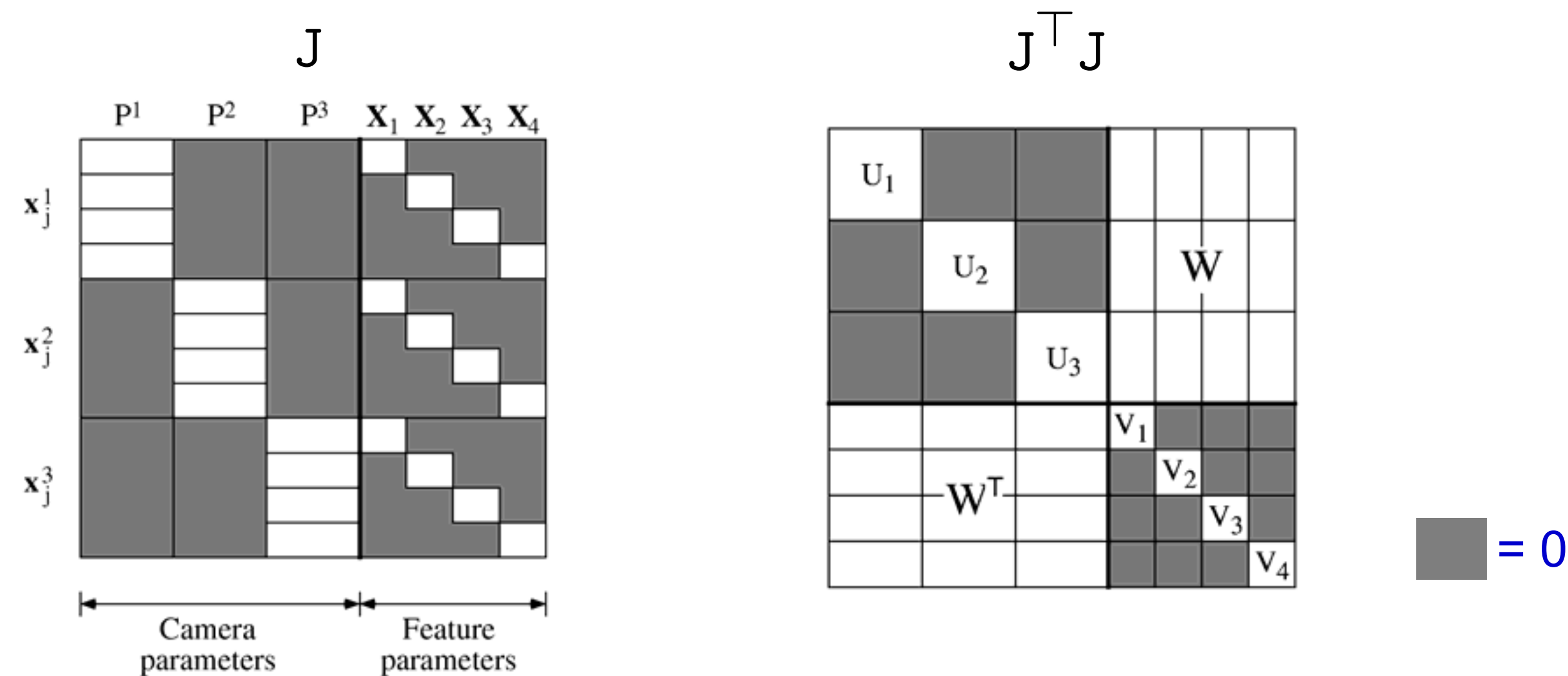
Intolerably large square matrices for some optimizations (e.g. neural networks with millions of variables)

Non-linear Least Squares

$$\min_{\mathbf{x}} C(\mathbf{x}) \quad C(\mathbf{x}) = (f(\mathbf{x}) - \mathbf{b})^T (f(\mathbf{x}) - \mathbf{b})$$

$$\epsilon = \mathbf{b} - f(\mathbf{x}) \quad \Delta \mathbf{x} = (\mathbf{J}^T \mathbf{J} + \lambda \text{diag}(\mathbf{J}^T \mathbf{J}))^{-1} \mathbf{J}^T \mathbf{e}$$

Q: Doesn't large-scale SfM have many variables? 3D points, cameras...



Can use sparse structure for efficient LM updates

Questions?